REPORT • WEEK 6

WEB SECURITY & VULNERABILITY TESTING

Vulnerability Assessment and Penetration Testing (VAPT) — Practical Assessment Report

PRACTICALS COVERED

SQL Injection • Reflected XSS • Stored XSS
Burp Suite Traffic Analysis • OWASP ZAP Scanning

SUBMITTED BY

Amar Pratap

Organization: DG Interns Hub
Environment: Local DVWA lab in VMware

REPORT DATE

14 OCT 2026

AUTHORIZED TRAINING ENVIRONMENT ONLY

Evidence target: http://127.0.0.1:4280

WEB SECURITY & VULNERABILITY TESTING

Week 6 Practical Report

This document records the practical activities visible in the supplied evidence: testing of intentionally vulnerable DVWA modules, HTTP traffic inspection with Burp Suite Community Edition, and automated scanning with OWASP ZAP. The structure follows the example report shared by the user, with clearer explanations, evidence captions, findings, and mitigation guidance.

Field	Details
Submitted by	Amar Pratap
Organization	DG Interns Hub — Cybersecurity Internship
Lab platform	VMware Workstation with a Debian/Kali-style Linux guest environment
Target	http://127.0.0.1:4280 (localhost only)
Tools evidenced	DVWA, browser, Burp Suite Community Edition, OWASP ZAP 2.17.0
Report evidence date	ZAP HTML report screenshot shows 14 October 2026

Important scope note: This is a training-lab assessment. The screenshots demonstrate behavior in the local DVWA environment and do not establish that any public or production website was tested.

TABLE OF CONTENTS

This report is organized so a reviewer can follow the test environment, method, evidence, findings, and recommended fixes.

No.	Section	Page
1	Introduction	4
2	Project Objectives	5
3	Scope of Assessment	6
4	Tools and Technologies Used	7
5	Methodology	8
6	Task 1 — DVWA Environment and Access	9–10
7	Task 2 — SQL Injection Testing	11–12
8	Task 3 — Reflected XSS Testing	13–14
9	Task 4 — Stored XSS Testing	15–16
10	Task 5 — Burp Suite Traffic Analysis	17–18
11	Task 6 — OWASP ZAP Scanning	19–20
12	Findings and Security Observations	21
13	Risk Analysis and Recommendations	22–23
14	Conclusion and References	24

Evidence handling: screenshots are reproduced from the images supplied in this conversation. Captions describe what is visible; any issue requiring further validation is explicitly marked as such.

Appendix A · Original screenshot gallery 25·29

1. INTRODUCTION

Web applications accept input, process requests, communicate with databases, and return content to users. If input is handled unsafely or security controls are missing, an application may expose data, allow script execution in a browser, or disclose unnecessary technical information.

Vulnerability Assessment and Penetration Testing (VAPT) is a structured process used to identify, validate, prioritize, and help remediate security weaknesses. A good assessment combines manual testing, traffic inspection, automated scanning, evidence collection, and a clear explanation of the limits of the results.

For this practical, Damn Vulnerable Web Application (DVWA) was used as the intentionally vulnerable target. The supplied screenshots show the target at **http://127.0.0.1:4280**, which is a loopback address accessible from the same host/guest environment. SQL Injection and XSS demonstrations were performed in DVWA, while Burp Suite and OWASP ZAP were used to inspect traffic and identify additional security observations.

The purpose is educational: to understand how vulnerabilities occur and how developers can prevent them. Findings from a deliberately vulnerable application should not be interpreted as proof that an unrelated real-world system has the same weaknesses.

Assessment summary

Activity	Purpose
Manual testing	Observe how DVWA responds to normal and modified input.
Proxy analysis	Inspect HTTP requests, parameters, cookies, status codes, and response headers.
Automated scanning	Collect ZAP alerts for potential configuration and security issues.
Remediation planning	Map each confirmed issue or observation to a practical defensive control.

2. PROJECT OBJECTIVES

The objectives of this practical were to:

- Understand the purpose of web application security testing and the VAPT lifecycle.
- Access the DVWA training application in a controlled local environment.
- Demonstrate SQL Injection behavior using the dedicated DVWA module.
- Compare reflected and stored Cross-Site Scripting (XSS) behavior.
- Capture and inspect HTTP requests and responses with Burp Suite Community Edition.
- Run an OWASP ZAP scan and interpret the report's risk and confidence summaries.
- Differentiate scanner alerts from manually confirmed vulnerabilities.
- Document security impact, practical remediation measures, and retesting steps.

Expected learning outcome

By completing these activities, a learner should be able to explain why unsafe SQL query construction and unencoded browser output are dangerous, describe how an intercepting proxy helps testing, and interpret automated scan output without overstating what it proves.

Professional reporting principle

A finding should state the affected component, evidence, expected versus observed behavior, impact, confidence, and a fix. If a scanner alert has not been manually validated, it should remain a potential issue rather than being reported as confirmed.

3. SCOPE OF ASSESSMENT

The assessment was limited to the intentionally vulnerable DVWA application visible in the supplied screenshots. The browser and ZAP evidence show the loopback target **http://127.0.0.1:4280**. Burp screenshots show HTTP traffic for the same local application.

In-scope activities

- Review the DVWA home page and access its vulnerability-training modules.
- Observe SQL Injection behavior in the DVWA SQLi module.
- Observe reflected and stored XSS test dialogs in the corresponding DVWA modules.
- Inspect HTTP requests, response status codes, and response headers in Burp Suite.
- Generate and review an OWASP ZAP HTML report for the local target.
- Summarize potential risks and recommend secure development controls.

Out of scope

- Testing any public website, external IP address, or production service.
- Credential theft, persistence, data exfiltration, or destructive testing.
- Claiming that all possible vulnerabilities were found or that remediation was verified.
- Treating informational scanner output as a confirmed exploitable vulnerability without validation.

Scope and safety

DVWA is designed to be vulnerable. Keep the lab isolated and bound to localhost or a private test network. Do not expose it to the public internet. Only test systems for which explicit authorization has been granted.

4. TOOLS AND TECHNOLOGIES USED

Tool / component	Role in the practical	Notes from evidence
DVWA	Intentionally vulnerable web app used as the target	Home page and SQLi/XSS module behavior are visible
Linux guest in VMware	Isolated environment for running the lab tools	Screenshots show VMware Workstation and a Linux g
Web browser	Access the local DVWA pages and display test results	Target URL shown as 127.0.0.1:4280.
Burp Suite Community Edition	Intercept and inspect HTTP traffic.	Proxy and HTTP History views are shown.
OWASP ZAP	Automated scanning and HTML report generation	ZAP version 2.17.0 appears in the report.
HTML report	Summarize ZAP alerts and supporting details	Saved locally under /home/amar in the screenshot.

What each tool contributes

DVWA provides repeatable vulnerable examples. **Burp Suite** gives visibility into what the browser sends and what the server returns. **OWASP ZAP** automates checks and organizes possible issues into alerts. These tools complement each other; none replaces manual validation or secure code review.

Version note

The ZAP report screenshot identifies version 2.17.0. The report records only the versions and configuration that can be supported by the supplied evidence.

5. METHODOLOGY

The practical followed a repeatable workflow designed for a controlled training target.

Step	Action	Why it matters
1. Confirm scope	Confirm the local DVWA URL and keep testing limited to that host.	Prevents accidental testing of unrelated systems.
2. Baseline	Open the relevant DVWA module and observe normal behavior.	Provides a reference for comparison.
3. Manual test	Submit a controlled test input in the designated DVWA module.	Shows whether input changes application behavior.
4. Inspect traffic	Review requests and responses in Burp Suite.	Reveals parameters, cookies, status codes, and headers.
5. Automated scan	Run ZAP against the local application and generate an HTML report.	Safe to add additional issues that may need review.
6. Analyze	Separate confirmed behavior from scanner observations.	Avoids false positives and overclaiming.
7. Recommend	Map each issue to a defensive control and a retest.	Makes the report actionable.

Evidence limitations

The screenshots document selected points in the workflow, not a full recording of every action. The report therefore explains the visible results and identifies where manual confirmation or retesting would be required.

6. TASK 1 — DVWA ENVIRONMENT AND ACCESS

DVWA is a deliberately vulnerable PHP/MySQL application created for security training. It provides modules that demonstrate common weaknesses in a safe, repeatable setting. The supplied home-page screenshot displays the DVWA warning that the application must not be placed on a public-facing server.

Environment observed

- VMware Workstation is visible around the Linux guest desktop.
- The application is accessed through a loopback address: http://127.0.0.1:4280.
- The DVWA navigation includes SQL Injection, SQL Injection (Blind), Reflected XSS, Stored XSS, and other training modules.
- The target is local to the test environment; no external host is shown as the assessment target.

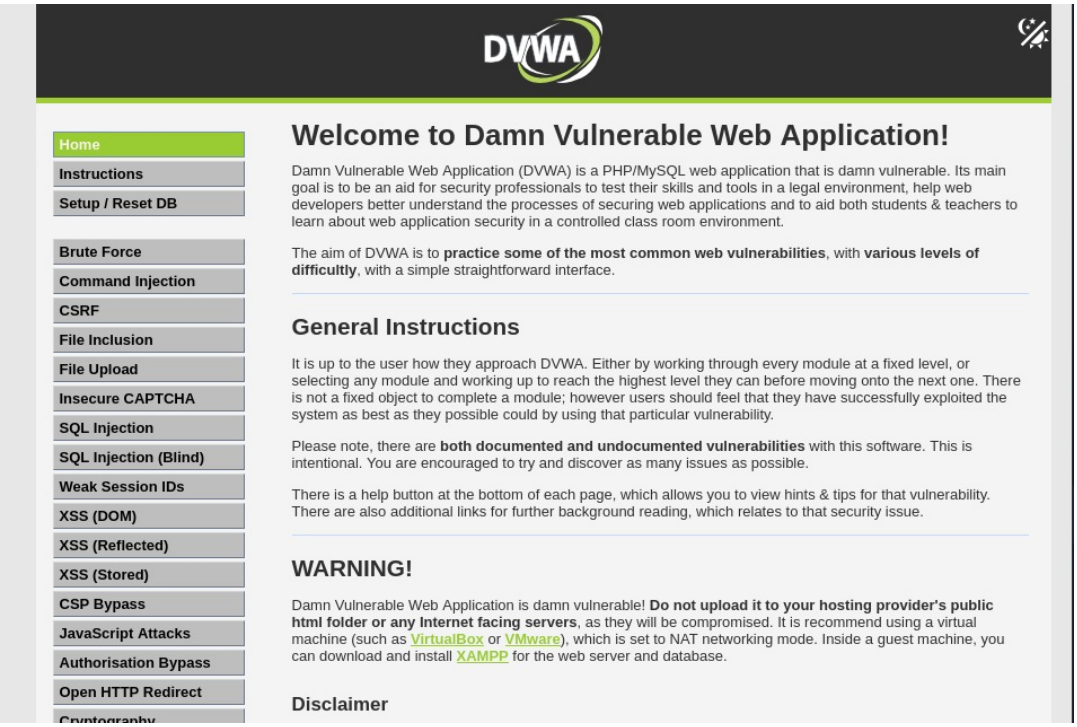


Figure 1. Original DVWA home page showing the training modules and security warning.

Result: The supplied evidence shows that DVWA was accessible locally and that the relevant vulnerability modules were available for testing.

TASK 1 — BASELINE AND LAB SAFETY

Before testing a vulnerability, record the normal application behavior. A baseline helps distinguish expected functionality from a change caused by test input. For example, the SQL Injection module accepts a user ID and normally returns the record associated with that ID.

Safe operating checklist

- Use the lab only for authorized training.
- Keep the application bound to localhost or a private, isolated network.
- Use harmless proof-of-concept inputs and avoid destructive actions.
- Do not copy session cookies, tokens, or credentials into a public report.
- Keep original evidence files and note the target URL and tool versions.
- After testing, reset the lab data if required by the training instructions.

Why a baseline matters

A single response is rarely enough to prove a vulnerability. Compare normal input with controlled test input, then inspect the resulting page and HTTP response. Record what changed and whether the behavior can be repeated.

Interpretation note

The screenshots establish access to the local application and the use of its modules. They do not, by themselves, establish the security posture of any other application.

7. TASK 2 — SQL INJECTION TESTING

SQL Injection (SQLi) occurs when an application constructs a database query by mixing untrusted input directly into SQL syntax. If the application does not separate data from query structure, an input may change the meaning of the query.

Evidence from the source view

The supplied source screenshot shows the DVWA SQL Injection source handling an incoming user ID and using it in a database query. The visible query structure includes the user-controlled ID in the WHERE clause. This is the type of construction that should be replaced with a parameterized query.

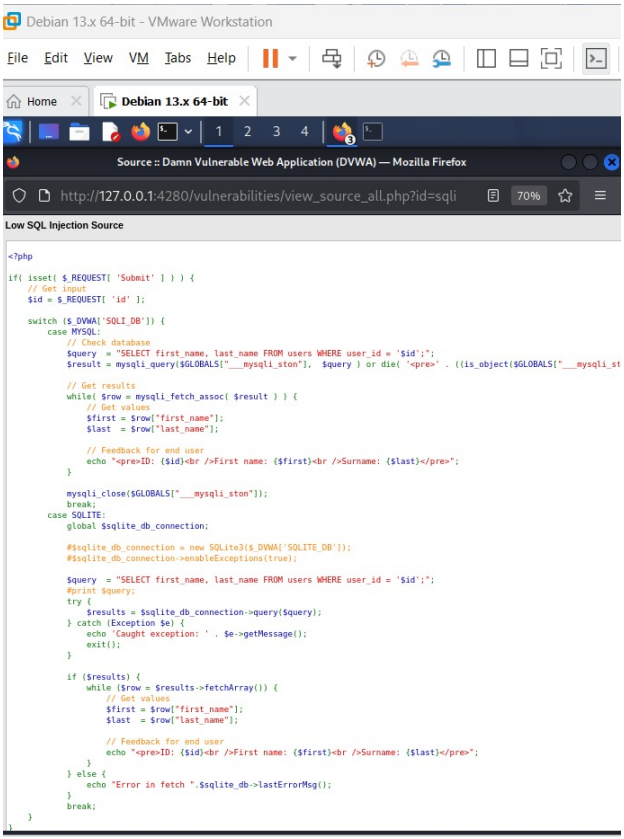


Figure 2. Original screenshot of the DVWA SQL Injection source view.

Security lesson: user input should be passed as a bound parameter, not concatenated into query text. Input validation is useful, but parameterization is the primary protection against SQL Injection.

TASK 2 — OBSERVED RESULT AND IMPACT

The screenshots show a baseline lookup returning a single record for a normal user ID. A modified test input then causes the training application to display multiple user records. This difference demonstrates that the input can influence the SQL query logic in the intentionally vulnerable DVWA module.

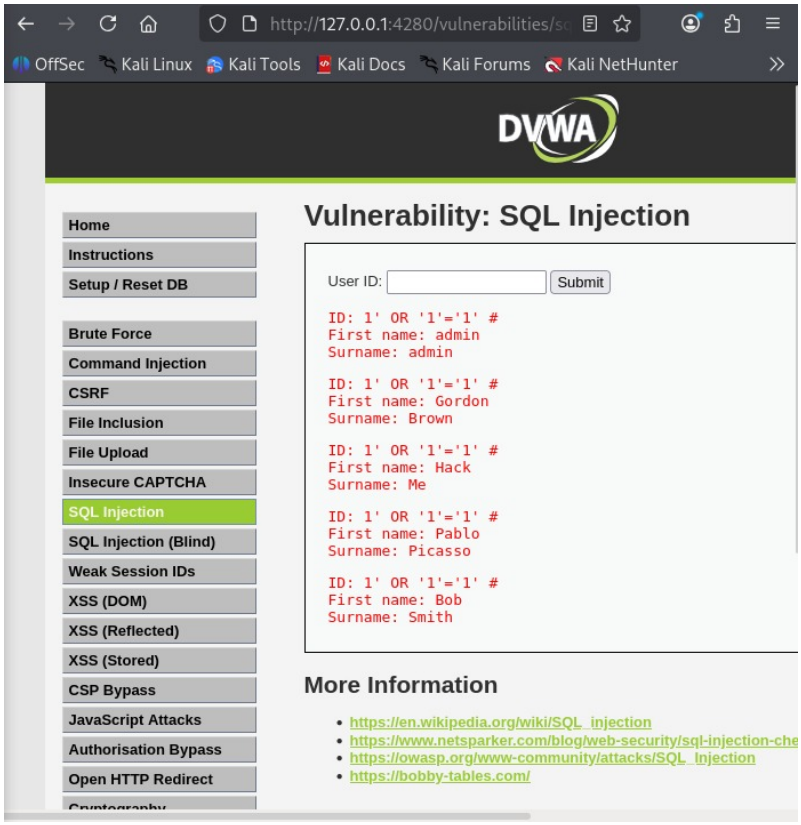


Figure 3. Original screenshot showing multiple records returned by the SQL Injection test.

Potential impact

- Unauthorized disclosure of database records.
- Authentication or authorization logic may be bypassed in affected applications.
- Depending on query permissions and database configuration, data could potentially be modified or deleted.

Recommended mitigation

Use prepared statements / parameterized queries, enforce input types and length limits, apply least privilege to database accounts, and avoid exposing database errors to users. Retest with normal and malformed inputs after fixing the query construction.

Result status: Demonstrated in the DVWA training target. This is expected behavior for an intentionally vulnerable lab, not a claim about a production website.

8. TASK 3 — REFLECTED XSS TESTING

Cross-Site Scripting (XSS) occurs when untrusted content is interpreted as executable script in a user's browser. Reflected XSS commonly involves input that is returned immediately in the response without suitable context-aware output encoding.

Observed evidence

The supplied browser screenshot displays an alert dialog labelled “XSS Test” on the local DVWA Reflected XSS page. This is a harmless proof-of-concept used to show that the test input was interpreted by the browser in the vulnerable training module.

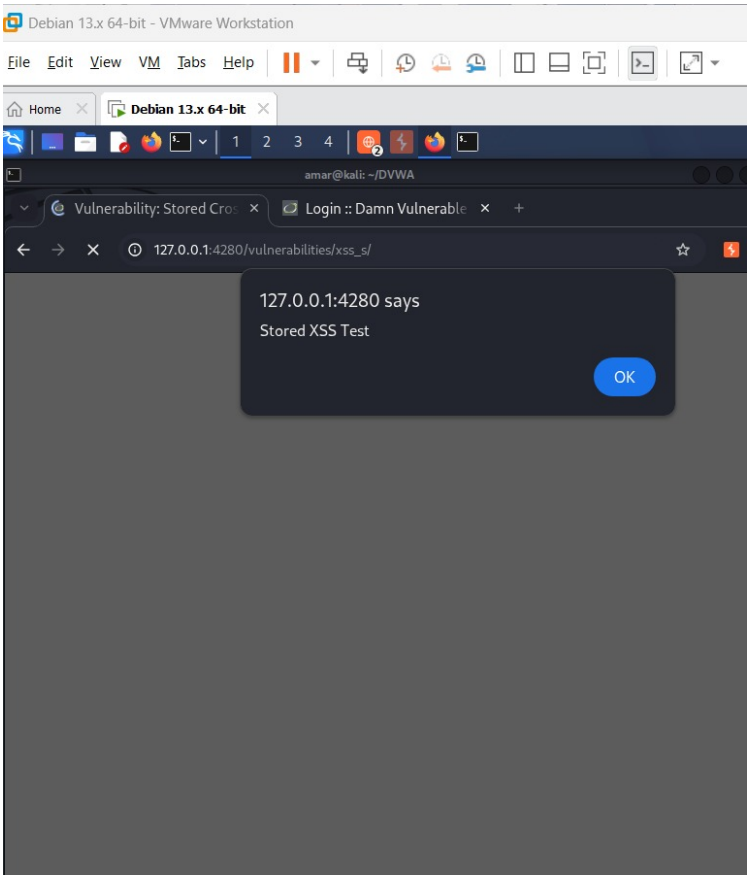


Figure 4. Original screenshot showing the reflected XSS test dialog.

Potential impact

On a real vulnerable application, reflected XSS could allow a crafted link or request to execute script in a victim's browser context. Impact depends on the affected page, browser context, session controls, and what sensitive actions the application exposes.

TASK 3 — REFLECTED XSS MITIGATION

A reflected XSS finding should be validated by checking the exact input location, response context, and browser behavior. An alert dialog in DVWA demonstrates the lab's vulnerable behavior, but it is not evidence that any other site is vulnerable.

Recommended controls

- Encode output for the correct context: HTML text, attributes, JavaScript, URL, or CSS contexts require different handling.
- Prefer safe DOM APIs such as textContent instead of inserting untrusted data as HTML.
- Validate input according to expected business rules, but do not rely on input filtering alone.
- Use a carefully designed Content Security Policy (CSP) as defense in depth.
- Retest with benign strings containing special characters and confirm they are displayed as text.

Validation criteria

After remediation, the same test string should appear as inert text and must not trigger script execution. Check the server response and rendered DOM, then repeat the test for every relevant input and output context.

Result: Reflected XSS behavior was demonstrated in the local DVWA training module, as shown by the supplied evidence.

9. TASK 4 — STORED XSS TESTING

Stored XSS occurs when user-controlled content is saved by an application and later rendered in a browser without appropriate encoding or sanitization. Unlike reflected XSS, the input may persist and affect later page views.

Observed evidence

The supplied screenshot shows an alert dialog labelled “Stored XSS Test” on the DVWA Stored XSS module. The screenshot supports that the stored-XSS proof-of-concept was triggered in the local training environment.

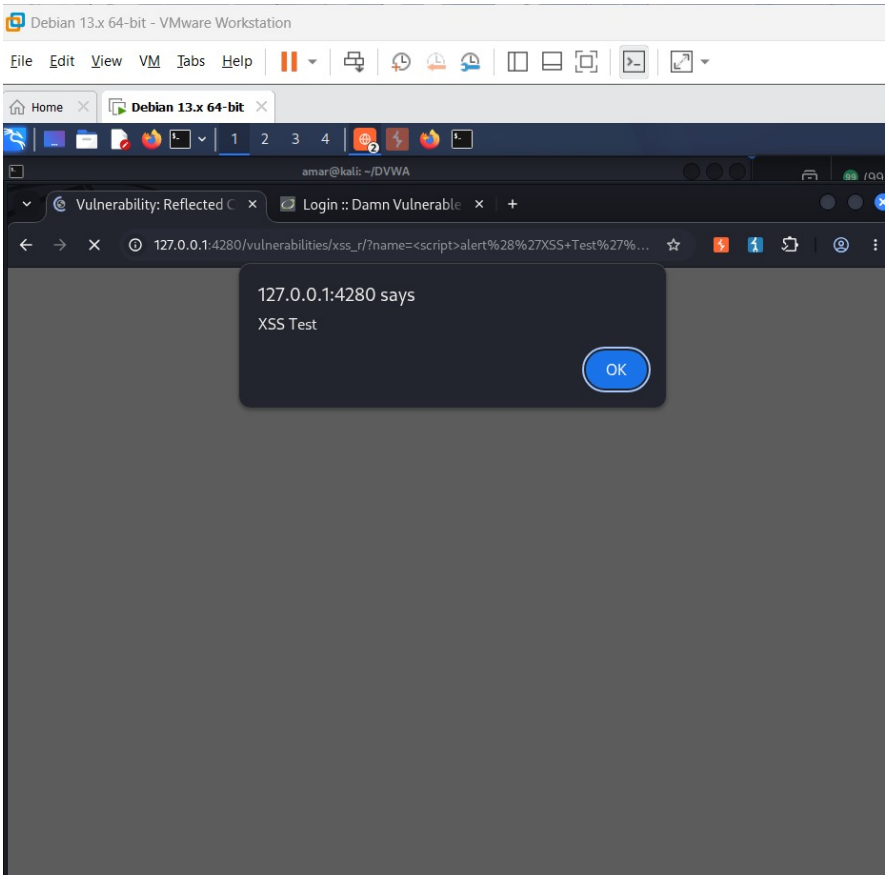


Figure 5. Original screenshot showing the stored XSS test dialog.

Potential impact

If similar behavior existed in a real application, stored script could execute for every user who views the affected content. This can increase exposure because the payload remains in stored application data until removed or neutralized.

TASK 4 — STORED XSS MITIGATION

Stored content must be treated as untrusted even when it has been saved in the application's database. Data can become dangerous when rendered in a browser context, so encoding at output time is essential.

Recommended controls

- Apply context-aware output encoding whenever stored values are displayed.
- If rich text is required, use a maintained HTML sanitizer with a restrictive allowlist.
- Use safe template defaults and avoid raw HTML insertion for user-generated content.
- Review stored records for existing unsafe content and remove or neutralize affected entries.
- Add CSP as a supporting control, not as a replacement for encoding and sanitization.

Retest criteria

Submit a harmless test string, revisit the page in a new session or browser context, and verify that it is rendered as text rather than executed. Confirm that the stored value is not still active in any page where it appears.

Result: Stored XSS behavior was demonstrated in DVWA as documented by the supplied screenshot. No claim is made that remediation was performed or verified.

10. TASK 5 — BURP SUITE WEB TRAFFIC ANALYSIS

Burp Suite Community Edition acts as an intercepting proxy between the browser and the target web application. It allows a tester to inspect requests and responses, including the HTTP method, path, query parameters, cookies, request headers, response status, and response headers.

The supplied screenshots show Burp Suite configured with a local proxy listener on port 8080 and traffic to DVWA at 127.0.0.1:4280. The Proxy / Intercept and HTTP history views are visible.

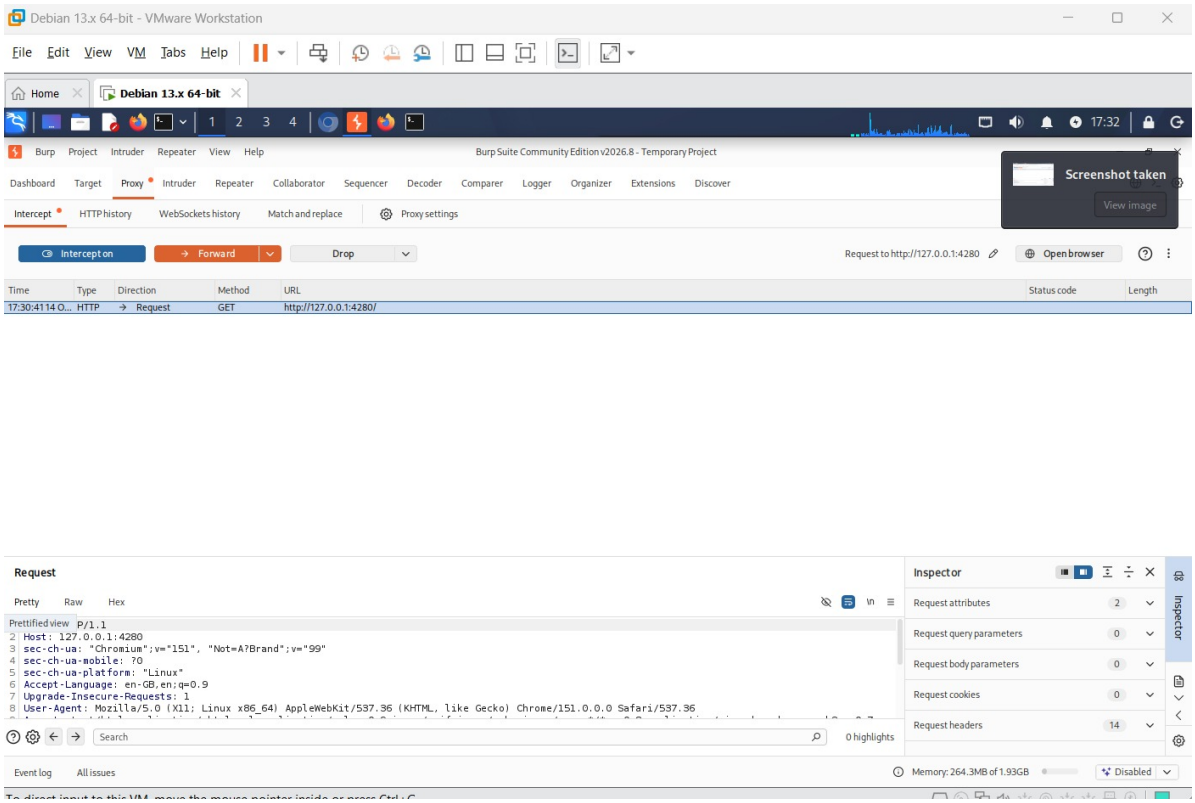


Figure 6. Original screenshot of Burp Suite traffic inspection during local DVWA testing.

Why this matters

Traffic inspection helps a tester determine where input is transmitted, whether it appears in a query string or request body, how cookies are sent, and what the server returns. It supports manual validation and helps preserve reproducible evidence.

TASK 5 — REQUEST / RESPONSE INTERPRETATION

The supplied Burp screenshots show requests for the local application, including navigation to the SQL Injection module and a login request. A later screenshot shows an HTTP 200 response for a SQLi request and a separate 302 redirect response.

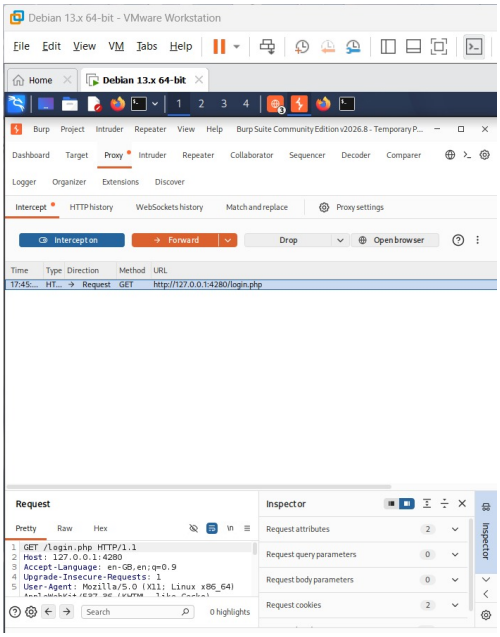


Figure 7. Original Burp screenshot showing request and response details.

How to interpret the visible fields

- **Request method and path:** identify the endpoint being accessed.
- **Parameters:** reveal where test input is sent; parameters should be validated by the application.
- **Cookies:** carry session state and should be protected and redacted in shared evidence.
- **HTTP status:** 200 means a response was returned; 302 usually indicates a redirect. Neither status alone proves a vulnerability.
- **Response headers:** can reveal configuration weaknesses or useful hardening opportunities.

Evidence hygiene

Before sharing Burp exports, remove session cookies, authorization headers, tokens, and any personal data. Use a clean test account and keep the evidence limited to the authorized lab.

11. TASK 6 — OWASP ZAP VULNERABILITY SCANNING

OWASP ZAP (Zed Attack Proxy) is a web application security testing tool that can crawl an application and run passive or active checks. Passive scanning evaluates observed traffic without deliberately changing requests; active scanning sends test requests and must only be used against systems where testing is authorized.

The supplied screenshots show OWASP ZAP version 2.17.0, the local DVWA target, the report-generation dialog, and a generated HTML report saved under /home/amar.

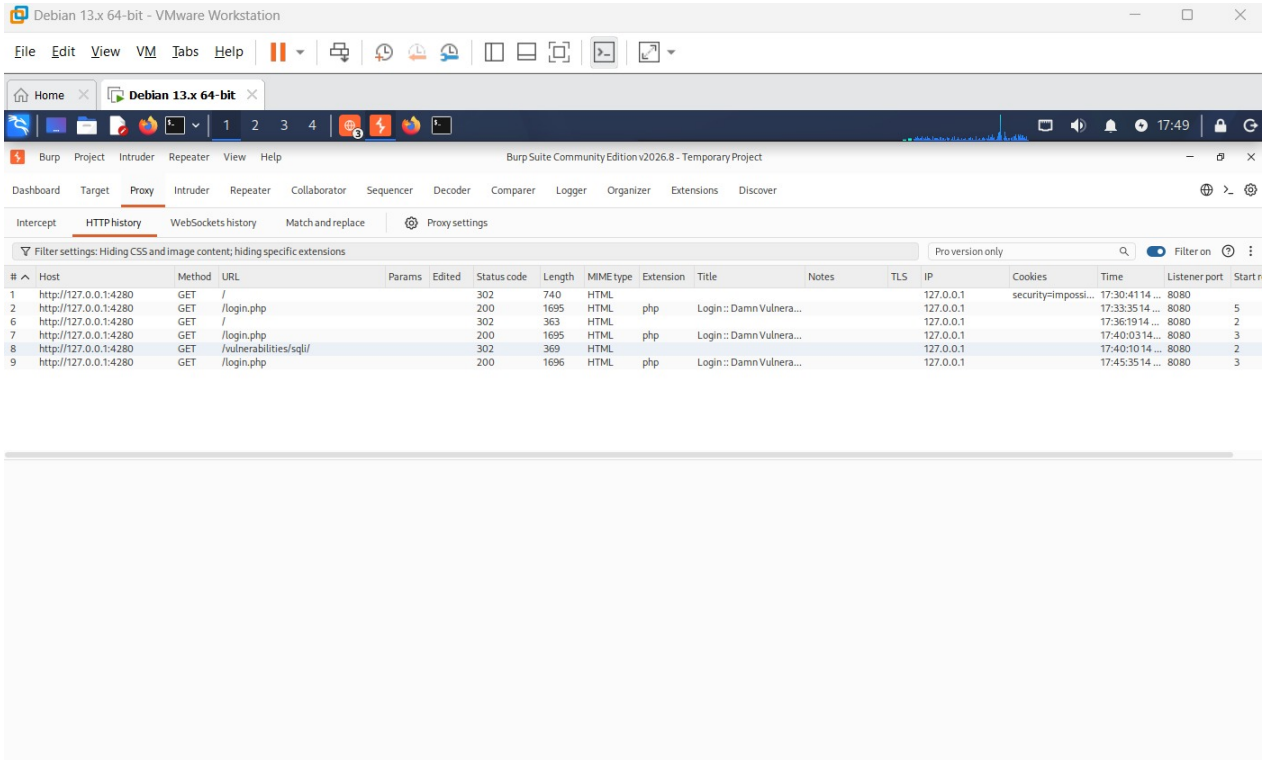


Figure 8. Original screenshot of the ZAP interface used for the assessment.

Scan scope visible in evidence

The report configuration lists **http://127.0.0.1:4280** as the target site. The report was generated locally and opened in a browser using a file:// URL. This supports the conclusion that the documented scan targeted the local training application.

TASK 6 — ZAP REPORT AND SUMMARY

The supplied ZAP HTML report includes an overview, summary tables, alert details, and references. Its summary groups alerts by risk and confidence, while the alert-type section lists observed alert categories and occurrence counts.

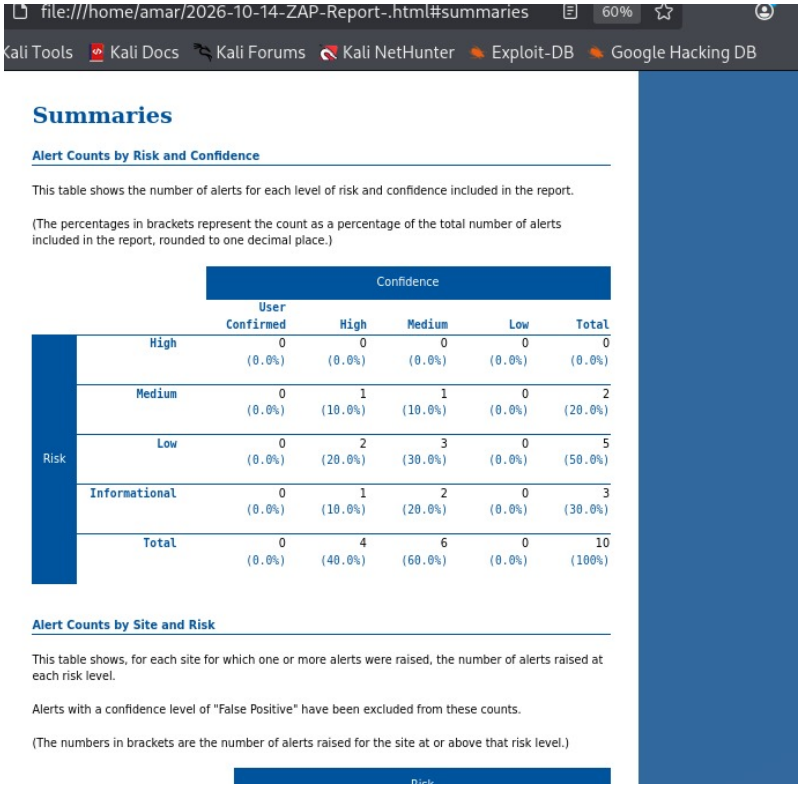


Figure 9. Original screenshot of ZAP's risk/confidence summary table.

The report summary shown in the evidence lists 10 alert records in its risk/confidence table: 2 medium-risk, 5 low-risk, and 3 informational; no high-risk alerts appear in that summary. The confidence summary shows 4 high-confidence and 6 medium-confidence entries.

Important interpretation

Alert counts can represent records or occurrences, not necessarily unique root causes. A single missing header may be reported on several responses. Treat each alert as a lead to validate, and do not equate the count directly with the number of independently exploitable vulnerabilities.

12. FINDINGS AND SECURITY OBSERVATIONS

The following summary separates the manually demonstrated behavior in DVWA from the potential configuration issues reported by ZAP. The DVWA SQLi/XSS tests are expected to succeed because the application is intentionally vulnerable.

Finding / observation	Evidence source	Assessment status	Security meaning
SQL Injection	DVWA SQLi module returns multiple records for modified input.	Demonstrated in lab	Input can affect query logic; use parameter
Reflected XSS	Browser alert shown on DVWA reflected XSS page.	Demonstrated in lab	Untrusted reflected content is interpreted b
Stored XSS	Browser alert shown on DVWA stored XSS page.	Demonstrated in lab	Stored content can execute when displaye
CSP header not set	ZAP report alert type; medium risk observation	Scanner observation; validate	CSP can reduce impact of some script-inje
Anti-clickjacking header missing	ZAP report alert type; medium risk observation	Scanner observation; validate	Framing controls help prevent clickjacking
SameSite cookie attribute missing	ZAP report alert type; low risk observation	Scanner observation; validate	Cookie cross-site sending behavior may n
Server/version disclosure and X-Content-Type-Options header missing	ZAP report alert type; low risk observation	Scanner observation; validate	Hardening opportunities; impact depends

The table reflects the evidence supplied and should not be interpreted as a full audit of every DVWA module or every possible vulnerability.

13. RISK ANALYSIS

Risk describes the potential harm if an issue is exploited, while confidence describes how strongly the evidence supports the alert. Severity should be assigned in context, considering exploitability, affected assets, required privileges, and business impact.

Category	What the evidence shows	Recommended interpretation
Manual SQL Injection	Modified input returns multiple database records in DVWA.	Confirmed demonstration in the intentionally vulnerable
Reflected XSS	A harmless test alert appears in DVWA.	Confirmed demonstration in the lab; real-world impact d
Stored XSS	A stored-XSS test alert appears in DVWA.	Confirmed demonstration in the lab; persistence can br
Medium-risk ZAP alerts	CSP missing and anti-clickjacking header issues appear in the report.	Validate each finding in the relevant pages; prioritize based o
Low-risk ZAP alerts	Cookie attribute, banner, server version, and content-type header observations appear.	Apply defense-in-depth hardening; avoid revealing versio
Informational ZAP alerts	Authentication request, session-management, and informational alerts appear.	Review context; informational alerts are not necessarily

No high-risk ZAP alert is shown in the supplied summary. This does not mean the application is secure: DVWA is intentionally vulnerable, and automated scans can miss issues or produce false positives.

13. RISK ANALYSIS AND RECOMMENDATIONS

Recommended remediation plan

Priority	Control	Verification step
1	Use prepared statements / parameterized queries for all database access.	Repeat SQLi tests; input must be treated as data ar
2	Apply context-aware output encoding and safe DOM APIs.	Repeat reflected and stored XSS tests; test strings
3	Use a restrictive, tested Content Security Policy where appropriate.	Inspect response headers; test policy in report-only
4	Add anti-framing protection with CSP frame-ancestors and/or X-Frame-Options.	Check headers and verify the application cannot be
5	Set suitable cookie attributes such as SameSite; review Secure and HttpOnly.	Inspect Set-Cookie headers and confirm behavior m
6	Set X-Content-Type-Options: nosniff and correct MIME types.	Inspect responses for the header and test expected
7	Minimize server banners and keep server/framework packages patched.	Check response headers and verify supported softw
8	Manually validate every ZAP alert and rerun the scan after changes.	Record false positives, confirmed findings, and resi

Retesting rule: a fix is not complete until the original test is repeated and the expected secure behavior is observed. The supplied screenshots document the initial testing and report generation; they do not show completed remediation or a post-fix retest.

14. CONCLUSION AND REFERENCES

Conclusion

This practical provided hands-on experience with web application security testing in a local DVWA environment. The supplied evidence demonstrates SQL Injection behavior, reflected and stored XSS proof-of-concepts, HTTP request/response inspection in Burp Suite, and automated scanning/report generation with OWASP ZAP.

The main learning is that secure applications must separate data from SQL query structure, encode untrusted output for the browser context, apply suitable HTTP security headers, protect session cookies, and keep software configurations up to date. Automated scanners are useful for discovery, but their alerts must be manually validated and prioritized in context.

The report is based on the original screenshots supplied for this task. It does not claim that any public website was tested, that all possible vulnerabilities were found, or that recommended fixes were implemented and retested.

References

1. OWASP, Web Security Testing Guide: https://owasp.org/www-project-web-security-testing-guide/

2. OWASP, SQL Injection Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

3. OWASP, Cross Site Scripting Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

4. OWASP ZAP documentation: https://www.zaproxy.org/docs/

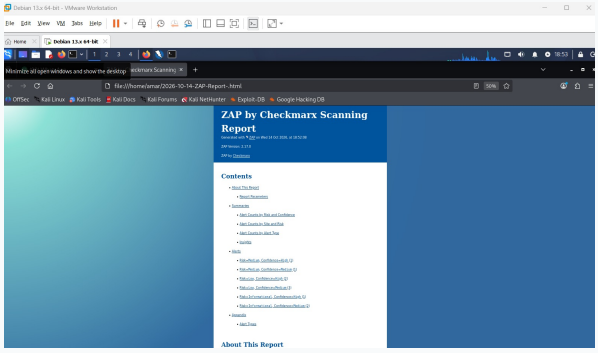
5. PortSwigger Web Security Academy: https://portswigger.net/web-security

6. Damn Vulnerable Web Application (DVWA): https://github.com/digininja/DVWA

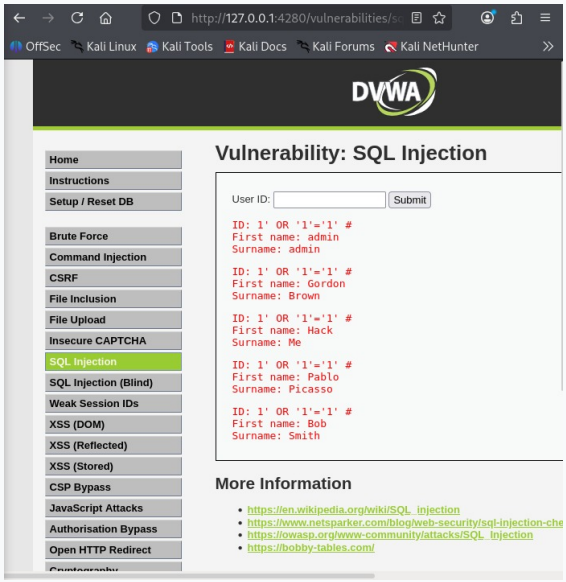
End of report

Appendix A — Original Screenshot Gallery

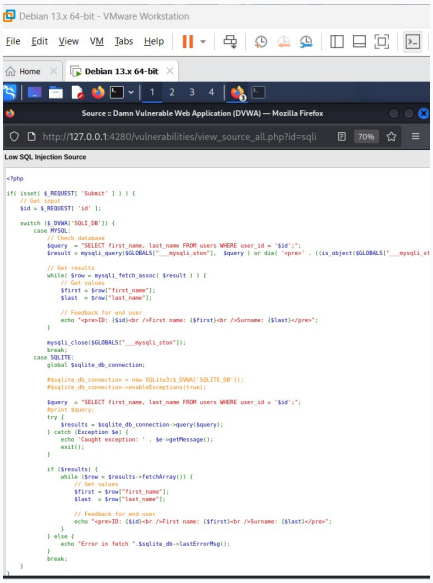
The following pages preserve the 19 original screenshots supplied for the report. Images are included as evidence; captions are descriptive and do not imply that every scanner alert is a confirmed vulnerability.



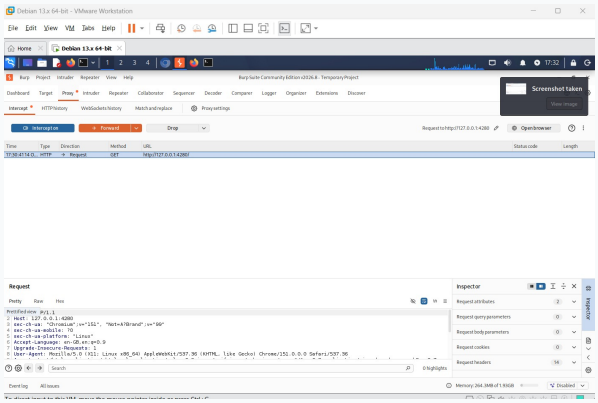
Original screenshot evidence 1



Original screenshot evidence 2



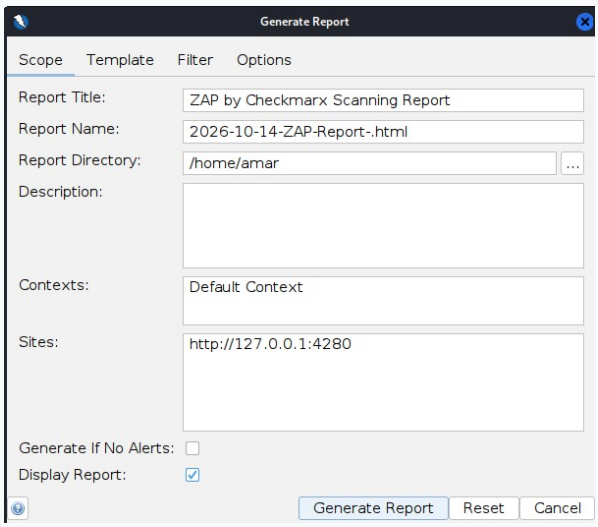
Original screenshot evidence 3



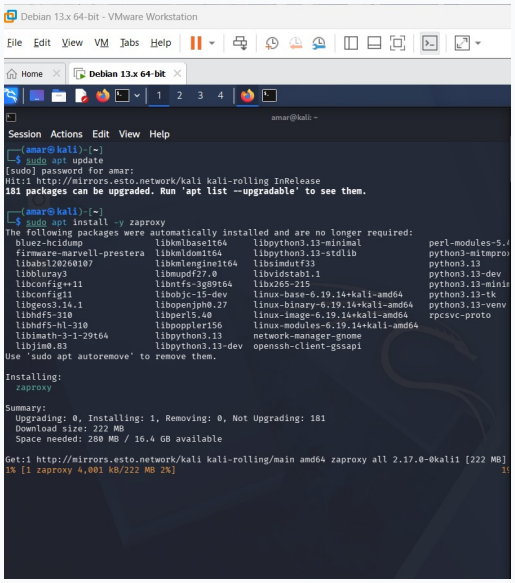
Original screenshot evidence 4

A1. Application and Manual Testing Evidence

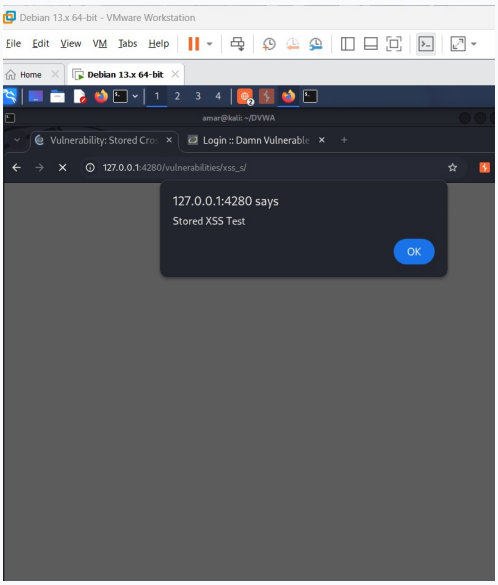
Screenshots related to the local DVWA application, source inspection, and manual SQL Injection testing.



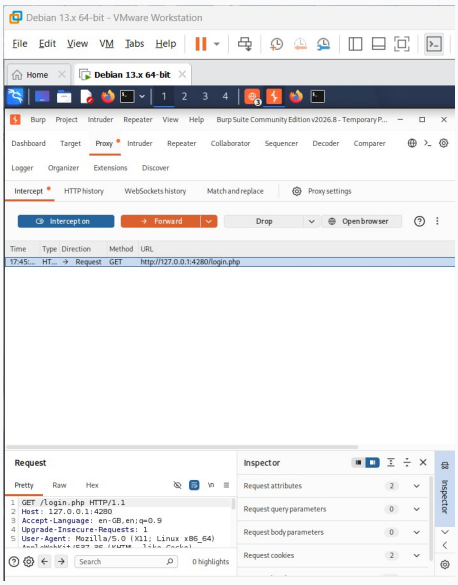
Original screenshot evidence 5



Original screenshot evidence 6



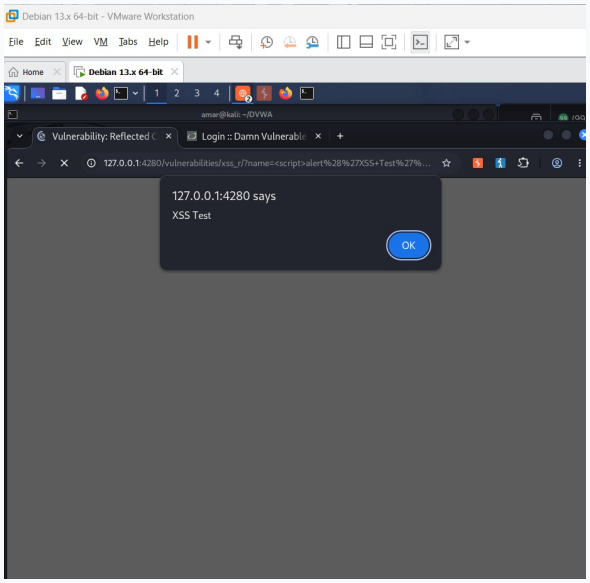
Original screenshot evidence 7



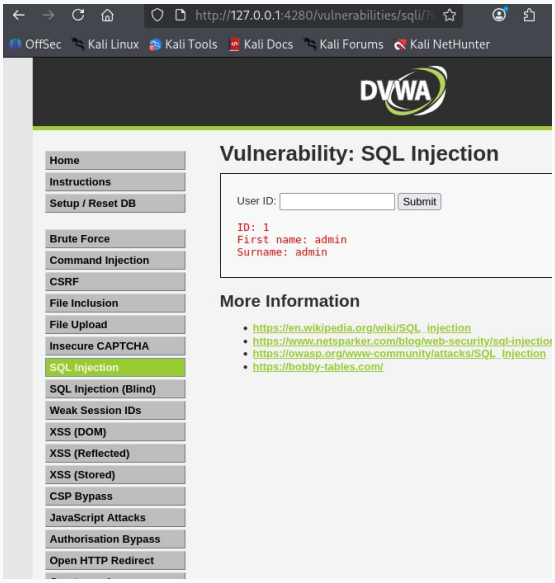
Original screenshot evidence 8

A2. Proxy and Browser Evidence

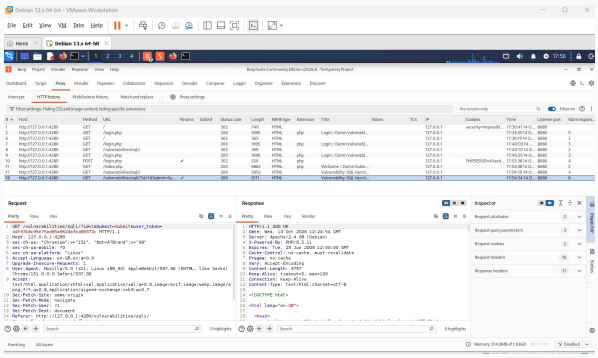
Screenshots related to Burp Suite traffic inspection and browser-visible proof-of-concept results.



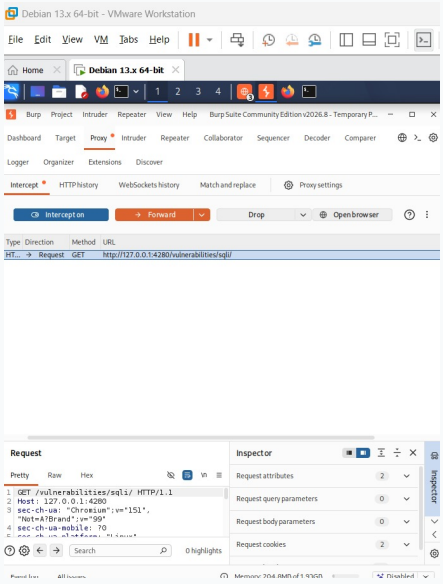
Original screenshot evidence 9



Original screenshot evidence 10



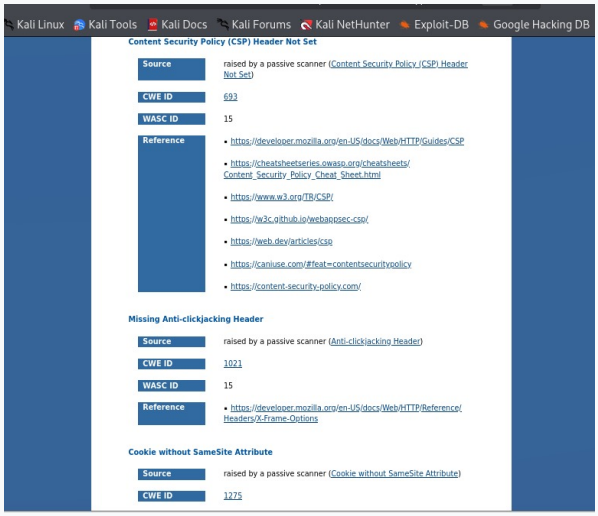
Original screenshot evidence 11



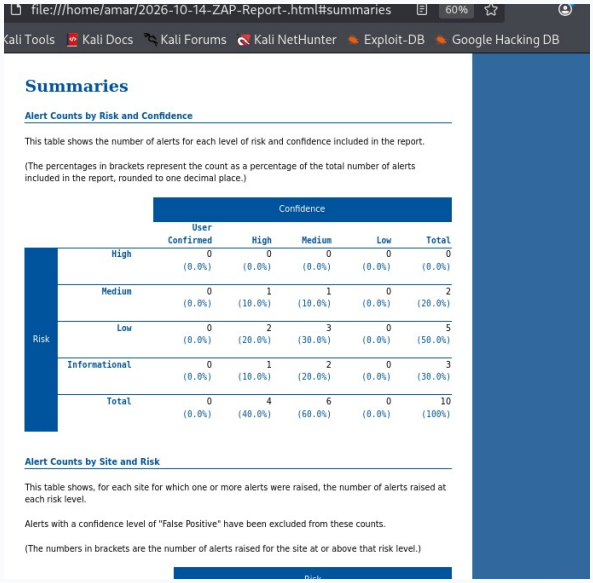
Original screenshot evidence 12

A3. ZAP Setup and Report Evidence

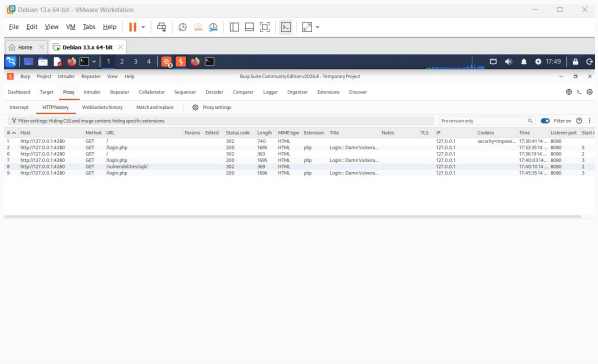
Screenshots related to ZAP setup, report configuration, generated HTML report, and alert summaries.



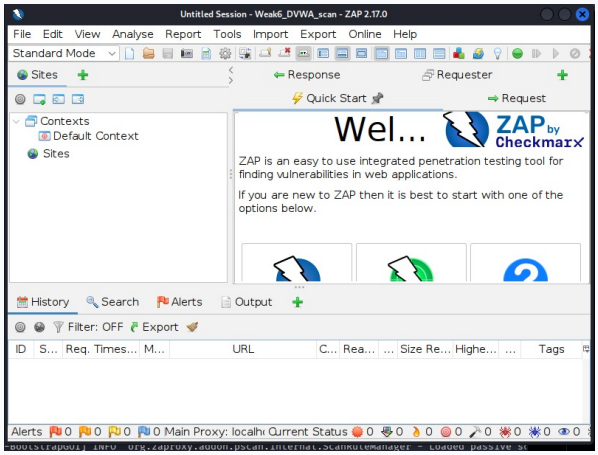
Original screenshot evidence 13



Original screenshot evidence 14



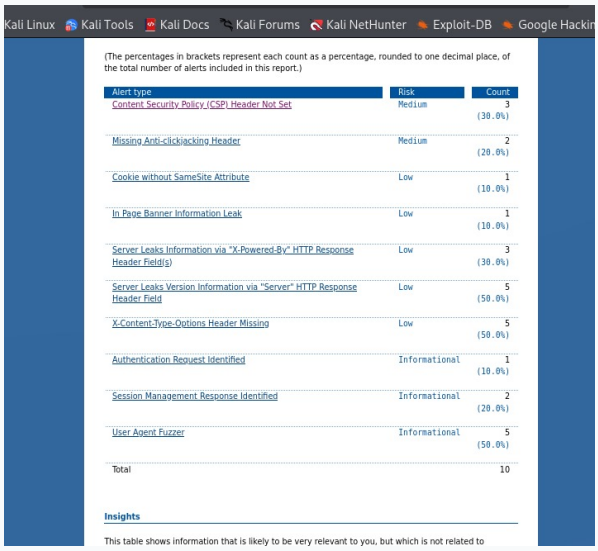
Original screenshot evidence 15



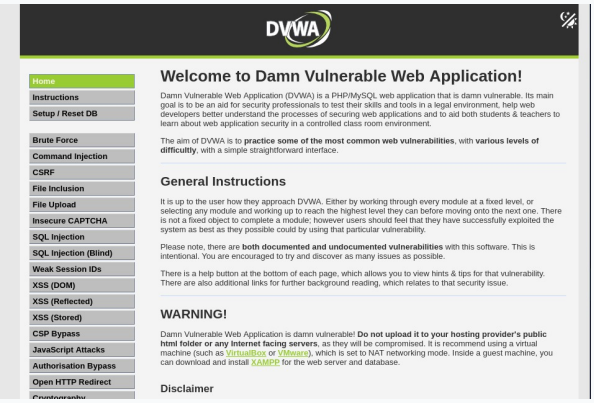
Original screenshot evidence 16

A4. Additional Original Evidence

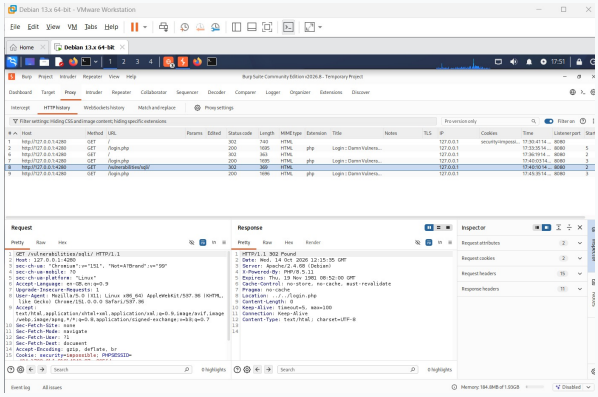
Remaining screenshots retained to preserve the full evidence set.



Original screenshot evidence 17



Original screenshot evidence 18



Original screenshot evidence 19